
GHULAM ISHAQ KHAN INSTITUTE

of Engineering Sciences and Technology

Faculty of Computer Science and Engineering

CE342L • Computational Methods and Tools Lab

Lab Report #8

Gauss–Seidel Iterative Method

Solving Linear Systems Numerically

Submitted by:	Muhammad Daim Bin Zakriya
Registration No:	2023405
Submitted to:	Sir Ali Husnain
Semester:	6th Semester

Contents

Objective	2
Theory	2
1 Task 01 — Gauss–Seidel Method (Diagonally Dominant)	3
Conclusion	7

Objective

The objective of this lab is to understand and implement the Gauss–Seidel Iterative Method for solving systems of linear equations in MATLAB. Additionally, the lab compares the Gauss–Seidel method with the Jacobi variant, and investigates how the immediate use of updated values leads to faster convergence than the Jacobi method.

By the end of this lab, students will be able to:

- Understand the mathematical formulation of the Gauss–Seidel iterative method.
- Implement the Gauss–Seidel method in MATLAB.
- Verify the diagonal dominance condition and predict convergence behaviour.
- Compare convergence speed against the Jacobi method.
- Visualize iteration convergence through plots.

Theory

► Gauss–Seidel Iterative Method

For a linear system $Ax = b$, the Gauss–Seidel method isolates each variable from its respective equation and immediately substitutes freshly computed values:

$$x_i^{(k+1)} = (1/a_{ii}) * (b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)})$$

Updated values are used as soon as they are computed within the same iteration. The method converges when the coefficient matrix A is strictly diagonally dominant:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \forall i$$

► Gauss–Seidel vs. Jacobi Method

The Gauss–Seidel method improves upon Jacobi by using updated values immediately:

Gauss–Seidel Method	Jacobi Method
The Gauss–Seidel method updates values immediately, using the latest computed values within the same iteration, leading to faster convergence.	The Jacobi method, in contrast, waits until all values have been computed before updating the next iteration.

This typically leads to faster convergence (fewer iterations) compared to Jacobi for the same system.

► Diagonal Dominance Check

Diagonally Dominant $ a_{ii} \geq \sum_{j \neq i} a_{ij} $ Gauss–Seidel converges Values settle to solution	NOT Diagonally Dominant $ a_{ii} < \sum_{j \neq i} a_{ij} $ Gauss–Seidel may diverge Values grow unboundedly
---	--

1 Task 01 — Gauss–Seidel Method (Diagonally Dominant)

► Question

Implement the Gauss–Seidel Iterative Method and verify convergence for the following diagonally dominant system. Use initial guess $x_0 = [0, 0, 0]^T$ and $n = 20$ iterations.

$$10x + 2y + z = 9$$

$$2x + 8y + z = 13$$

$$x + y + 5z = 7$$

► Theory / Manual Analysis

Diagonal Dominance Check:

Row	Condition	Pass?
Row 1	$ 10 > 2 + 1 = 3$	✓
Row 2	$ 8 > 2 + 1 = 3$	✓
Row 3	$ 5 > 1 + 1 = 2$	✓

Strictly diagonally dominant $\Rightarrow$ Gauss–Seidel will converge.

Gauss–Seidel Update Equations (each new value is used immediately):

$$x^{(k+1)} = (9 - 2y^{(k)} - z^{(k)}) / 10$$

$$y^{(k+1)} = (13 - 2x^{(k+1)} - z^{(k)}) / 8$$

$$z^{(k+1)} = (7 - x^{(k+1)} - y^{(k+1)}) / 5$$

Exact solution (by elimination): $x \approx 0.5246$, $y \approx 1.3661$, $z \approx 1.0219$.

► MATLAB Code

```
%lab8q1 2023405

x(1) = 0;
y(1) = 0;
z(1) = 0;

n = 20;

% Gauss-Seidel iterations use updated values immediately
for j = 1:n
    x(j+1) = (9 - 2*y(j) - z(j)) / 10;
    y(j+1) = (13 - 2*x(j+1) - z(j)) / 8;
    z(j+1) = (7 - x(j+1) - y(j+1)) / 5;
end

% Print iteration table
for j = 1:length(x)
```

```

        fprintf('Iteration %d\t: x = %.6f\t, y = %.6f\t, z = %.6f\n', ...
            j-1, x(j), y(j), z(j));
    end

    % Plot convergence
    figure;
    plot(0:n, x, '-ro', 'LineWidth', 2, 'DisplayName', 'x');
    hold on;
    plot(0:n, y, '-bs', 'LineWidth', 2, 'DisplayName', 'y');
    plot(0:n, z, '-gd', 'LineWidth', 2, 'DisplayName', 'z');
    grid on;
    xlabel('Iteration');
    ylabel('Value');
    title('Gauss-Seidel Method Convergence');
    legend('Location', 'best');

```

► Output

```

1 %lab0q1_2023405
2 x(1) = 0;
3 y(1) = 0;
4 z(1) = 0;
5
6 n = 20;
7
8 for j = 1:n
9     z(j+1) = (9 - 2*y(j) - z(j))/10;
10    y(j+1) = (13 - 2*x(j+1) - z(j))/8;
11    x(j+1) = (7 - x(j+1) - y(j+1))/5;
12 end
13
14 for j = 1:length(x)
15     fprintf('Iteration %d\t: x = %.6f\t, y = %.6f\t, z = %.6f\n', j-1, x(j), y(j), z(j));
16 end
17
18 figure;
19 plot(0:n, x, '-ro', 'LineWidth', 2, 'DisplayName', 'x');
20 hold on;
21 plot(0:n, y, '-bs', 'LineWidth', 2, 'DisplayName', 'y');
22 plot(0:n, z, '-gd', 'LineWidth', 2, 'DisplayName', 'z');
23 grid on;
24 xlabel('Iteration');
25 ylabel('Value');
26 title('Gauss-Seidel Method Convergence');
27 legend('Location', 'best');

```

Command Window

```

>> t3
Iteration 0 : x = 0.000000 , y = 0.000000 , z = 0.000000
Iteration 1 : x = 0.900000 , y = 1.400000 , z = 0.940000
Iteration 2 : x = 0.526000 , y = 1.376000 , z = 1.019600
Iteration 3 : x = 0.522840 , y = 1.366840 , z = 1.022064
Iteration 4 : x = 0.524426 , y = 1.366136 , z = 1.021888
Iteration 5 : x = 0.524584 , y = 1.366118 , z = 1.021860
Iteration 6 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 7 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 8 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 9 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 10 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 11 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 12 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 13 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 14 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 15 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 16 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 17 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 18 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 19 : x = 0.524590 , y = 1.366120 , z = 1.021858
Iteration 20 : x = 0.524590 , y = 1.366120 , z = 1.021858

```

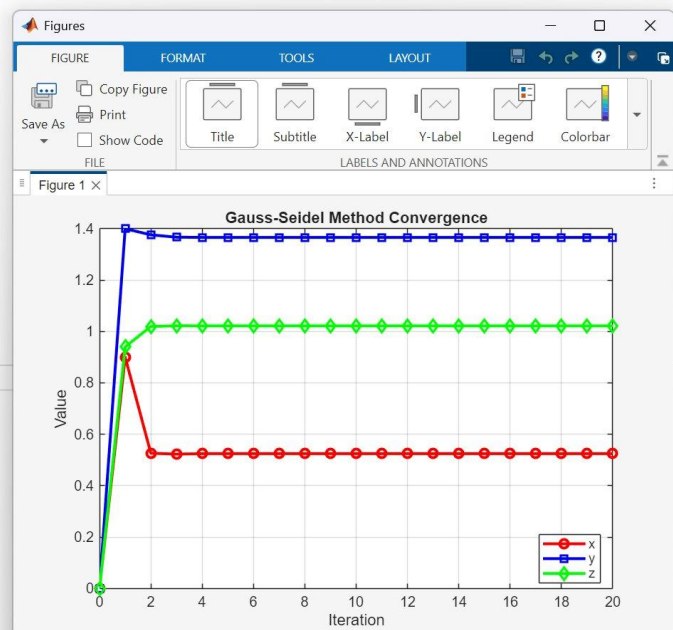


Figure 1: Task 01 — Gauss–Seidel convergence for diagonally dominant system (iterations + plot)

► Discussion

Converged successfully.

The iteration table shows all three variables stabilizing by approximately iteration 6–7:

$$x \approx 0.524590, \quad y \approx 1.366120, \quad z \approx 1.021858$$

The convergence plot shows rapid decay — variables reach their settled values far sooner than the Jacobi method (which required ~14–15 iterations for the same system). This is because Gauss–Seidel immediately passes the updated x into the y equation, and both updated x and y into the z equation, propagating improvements within a single pass. This confirms that the diagonal dominance condition guarantees convergence for the Gauss–Seidel method, and that Gauss–Seidel converges faster than Jacobi for the same system.

Conclusion

This lab demonstrated the Gauss–Seidel iterative method for solving a 3×3 linear system in MATLAB:

- Task 1 (Gauss–Seidel — Convergent): The diagonally dominant system converged to $(x, y, z) \approx (0.5246, 1.3661, 1.0219)$ after approximately 6–7 iterations. This is roughly half the number of iterations required by the Jacobi method for the same system, confirming the theoretical advantage of immediate value updates.

Key Takeaway: Always verify diagonal dominance before applying the Gauss–Seidel method. When $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for all rows, convergence is guaranteed. Compared to Jacobi, Gauss–Seidel converges faster because updated values are propagated immediately within each iteration rather than deferred to the next cycle.
